Build a Semester architecture blueprint: OIDC IdP federation, custom domain routing, and an integrated education app suite in one spec

Computer
Build a Semester product roadmap: phased milestones for building an education ecosystem with OIDC auth and an integrated app suite
Computer
Show me the OIDC and federation layer design
Detail the Semester Mail and JMAP storage plan
Draft the phase 1 to phase 3 rollout roadmap

Build Semester around a shared **platform** spine: one canonical account system, an OIDC issuer and school-federation broker, verified domain routing, tenant-aware authorization, and an integrated education app suite. Semester Mail should use a standards-based mailbox engine behind Semester-owned policy and user experience—not become a second identity system or a duplicate academic database.

This specification defines the OIDC/federation layer, domain routing, Mail/JMAP storage, and a gated phase 1–3 rollout. All names, routes, schemas, and operational targets below are proposed; they are not claims of deployed infrastructure.

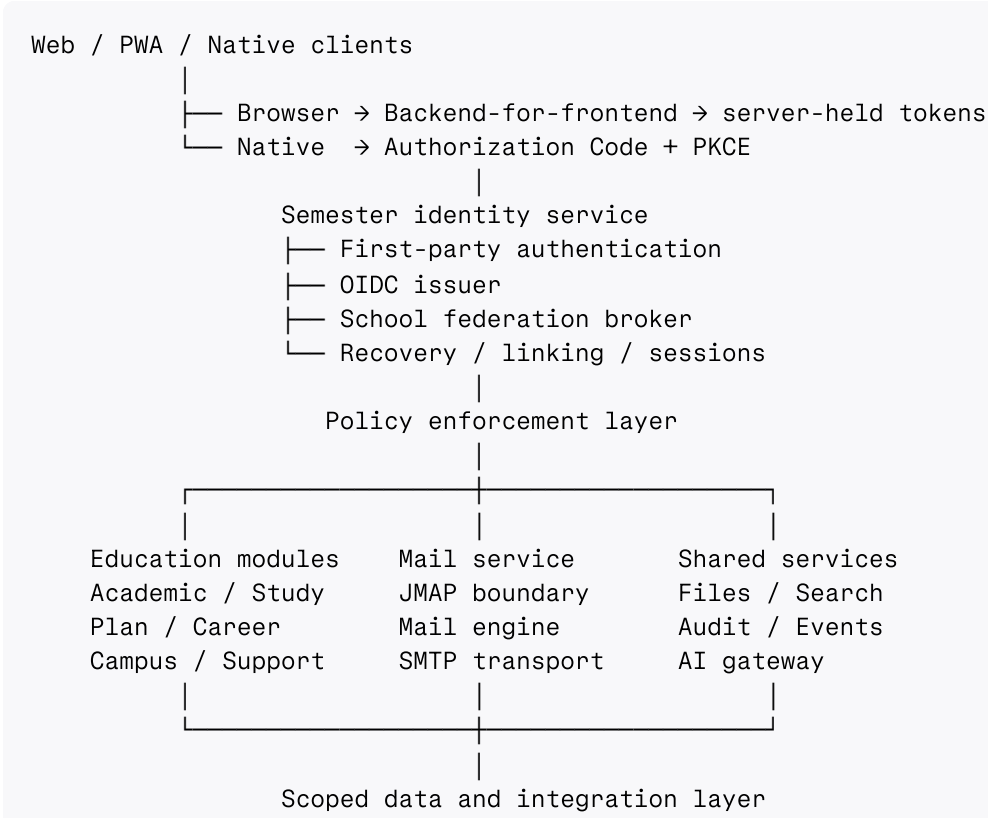
1. Architecture contract

Non-negotiable boundaries

1. One person can have one persistent Semester identity and multiple independently governed school memberships.
2. Authentication establishes identity; current policy establishes permission.
3. Personal data and institution-governed data have explicit, different ownership scopes.
4. Mail addresses, usernames, and school affiliations are attributes—not interchangeable identifiers.
5. Every app uses shared identity, policy, files, search, events, audit, and AI controls.
6. Integrations have reconciliation, rollback, and portability paths.
7. Routes and client-provided tenant IDs never grant authority.

Semester's existing role-system specification requires this canonical account/membership model and shared policy enforcement. Extend it rather than introduce parallel identity or permissions systems. [1]

Deployment topology



Keep education-domain modules in the existing modular architecture where practical. Mail transport and a hardened identity service may run separately because their operational and security requirements differ; a box in this diagram does not automatically require a microservice.

The supplied architecture describes a modular-monolith direction and shared platform components that were not yet fully adopted at its recorded snapshot. Reconcile with those components first. [2]

2. Custom domain and routing specification

Use a verified domain placeholder `D` until actual ownership is confirmed.

Host	Proposed responsibility	Routing constraint
`D`	Company/public website	No institutional data
`app.D`	Unified app and web backend-for-frontend	Host-scoped application sessions
`identity.D`	OIDC issuer and authentication	Fixed issuer; explicit clients and redirects
`account.D`	Account and security interface	Sensitive changes require step-up
`mail.D`	Mail interface	Same canonical identity; mailbox authorization
`status.D`	Operational status	No sensitive customer details

These are proposed namespaces, not verified DNS records.

Routing and security rules

- Maintain an allowlist of accepted hostnames.
- Reject unrecognized `Host` headers and untrusted forwarded-host values.
- Use exact registered OAuth redirect URIs, not arbitrary return URLs.
- Keep application cookies host-scoped; do not share a powerful session cookie across every subdomain.
- Obtain and monitor certificates for every activated hostname.
- Separate web routing, email MX records, and mail-authentication records in the domain inventory.
- Require approved DNS changes with recorded ownership and rollback instructions.

RFC 9700 provides the OAuth security baseline, including authorization-code protections and redirect handling.[3]

Institution-branded domains

A future host such as `learn.school.example` should require:

```
Institution authorization
→ DNS ownership challenge
→ verified tenant-to-host mapping
→ certificate issuance
→ routing and isolation tests
→ approved activation
```

A hostname selects a candidate tenant context. Membership and policy still determine access.

Removing a custom domain must revoke its mapping and prevent stale routing or takeover; it must not delete the underlying institution's records.

3. OIDC issuer and federation layer

3.1 Reconcile the existing authentication authority

Before selecting an identity engine, inspect the existing authentication implementation and account IDs.

Choose one documented outcome:

- Extend the existing authority if it meets issuer, federation, recovery, and lifecycle requirements.
- Put a supported federation/identity layer in front of it with an explicit subject-mapping contract.
- Migrate to a new authority through verified identity mapping and rollback.

Do not operate two unrelated identity databases and try to reconcile them by email address.

Use a maintained OIDC implementation. Semester should own configuration, policy, UX, and lifecycle—not invent token-signing and protocol-validation code from scratch.

3.2 Proposed issuer interface

```
Issuer:                    https://identity.D
Discovery:                 /.well-known/openid-configuration
Authorization endpoint:    /authorize
Token endpoint:            /token
Signing-key endpoint:      /jwks.json
UserInfo endpoint:         /userinfo

Additional supported functions:
  revocation
  session/logout handling
  optional token introspection
```

Endpoint paths other than the discovery convention are illustrative; publish the actual supported endpoints through discovery.

3.3 Client classes

Client	Authentication design
First-party web	Confidential client; backend-for-frontend
First-party native	Public client; system browser and PKCE
Approved partner web app	Separate registration, consent, audience, and scopes
Service integration	Separate workload identity; no impersonated human login

Use Authorization Code with PKCE `S256`. Public clients must use PKCE under the current OAuth security BCP; Semester should require it for its confidential clients too. Disable implicit and resource-owner-password flows.[3]

For browser applications, keep tokens server-side where the backend-for-frontend architecture permits. Current browser OAuth guidance identifies code flow with PKCE and refresh-token protections as central safeguards.[4]

3.4 Authentication transaction

```
1. Create transaction:
   client_id, redirect_uri, state, nonce, PKCE challenge,
   requested scopes, requested authentication assurance

2. Authenticate:
   Semester credential/passkey or approved school federation

3. Validate federation response, when applicable

4. Resolve canonical account and external identity mapping
```

```
5. Issue one-time authorization code

6. Exchange code; validate client and PKCE verifier

7. Validate ID token:
   signature, issuer, audience, expiry, nonce as applicable

8. Create application session

9. Resolve current workspace and resource permissions
```

OIDC defines the issuer, subject, audience, and authentication-response validation model.[5]

3.5 Federation connection record

Proposed logical fields:

```
connection_id
tenant_id
protocol: OIDC | SAML
trusted issuer / entity identifier
approved metadata source
client configuration or certificate references
claim-mapping version
required authentication assurance
provisioning authority
status: draft | tested | approved | active | suspended
configuration version and approving actor
```

Secrets belong in a secrets manager; the database stores references.

For OIDC, validate trusted issuer, signature, audience, nonce, and transaction correlation. For SAML, use a maintained validator with signature, audience, destination, timing, and request-correlation checks.

A domain hint or school selection helps route authentication. It does not prove school membership.

3.6 Identity and authorization records

Logical record	Constraint
Account	Immutable internal ID
External identity	Unique trusted connection + issuer + subject
Membership	Account + institution + lifecycle state
Role/capability assignment	Scoped and time-bounded where appropriate
Session	Authentication method, assurance, expiry, revocation
OAuth grant	Client, account, consented scopes, revocation

Logical record	Constraint
Security event	Actor, action, result, correlation ID; no raw tokens

For partner applications, use pairwise subject identifiers where appropriate to reduce cross-application correlation. Email remains an attribute; provider subject identifiers establish identity mappings. [6] [5]

3.7 Authorization and revocation

Do not embed every mutable institutional permission in a long-lived token.

For a consequential request:

```
Validate session/token
→ resolve account
→ verify active membership
→ resolve resource ownership and relationship
→ evaluate capability, policy, consent, entitlement
→ step-up or approval if required
→ execute controlled command
→ audit + event
```

On offboarding:

- Disable affected memberships and grants.
- Revoke relevant sessions and refresh-token families.
- Invalidate policy caches.
- Deny access using current server-side state.
- Preserve unrelated personal and institutional access.

Signing-key rotation needs overlap for legitimate token validation. It is not a substitute for targeted account revocation.

4. Semester Mail and JMAP storage plan

4.1 Select one mailbox source of truth

Recommended initial design:

> A qualified mail engine owns authoritative mailbox objects, message state, blobs, and submission state. Semester owns provisioning policy, identity mappings, entitlements, education-workflow references, and the native interface.

Do not build a second complete mailbox database in Semester alongside the engine.

JMAP Core defines synchronization and state handling; JMAP Mail defines objects including Email, Mailbox, Thread, Identity, and EmailSubmission. [7] [8]

4.2 Storage responsibilities

Layer	Authoritative content	Semester rule
Semester relational database	Mailbox ownership, engine mappings, aliases, entitlements, policy, abuse cases	No unnecessary full-message duplication
Mail engine metadata	Mailbox membership, flags/keywords, threads, message and submission state	Single source of truth
Engine-controlled blob storage	Original messages and attachment data	Private access; authenticated retrieval
Search index	Derived mailbox-scoped search projection	Rebuildable; independently access-controlled
Audit/event store	Provisioning, authorization, submission, support actions	Exclude message bodies by default
Client cache	Recently permitted message data and sync states	Revocable; encrypted where supported; clear on sign-out

An engine `blobId` is an opaque reference, not a public download URL or an authorization grant.

Avoid cross-tenant content deduplication unless its confidentiality and deletion implications are explicitly proven. Favor isolation over speculative storage savings.

4.3 Provisioning state machine

```
Requested
→ address reserved
→ policy/entitlement approved
→ engine mailbox created
→ identity/address mapping verified
→ inbound delivery tested
→ outbound submission tested
→ active

Failure:
→ failed/reconciling
→ retry safely or release reservation under policy
```

Use idempotency keys and a reconciliation worker. Do not mark a mailbox active merely because one provider API call returned success.

4.4 Inbound delivery

```
SMTP ingress
→ recipient validation
→ authentication/spam/malware evaluation
→ durable acceptance
→ engine message/blob commit
→ mailbox state update
```

```
→ index projection
→ permitted notification
```

Return successful acceptance only after the receiving infrastructure can durably retain responsibility for delivery.

Indexing or notifications may lag without making a successfully stored message disappear.

4.5 Outbound delivery

```
Compose/upload
→ verify mailbox rights and send-as identity
→ apply quota/abuse checks
→ create submission
→ queue transport
→ DKIM signing and SMTP attempt
→ delivery/bounce status
```

Distinguish:

- Accepted for submission.
- Queued.
- Accepted by recipient server.
- Failed or bounced.

Do not promise exactly-once external SMTP delivery. Use idempotent submission requests to reduce duplicate sends, while acknowledging transport ambiguity.

Configure and test SPF, DKIM, and DMARC; Gmail's sender requirements make authenticated delivery an activation requirement.[9]

4.6 JMAP synchronization

Persist engine-provided state values as opaque tokens.

```
Load Session capabilities
→ query/get initial permitted objects
→ store object/query state
→ request changes since stored state
→ apply creates, updates, destroys
→ continue pagination where required
```

If the engine returns `cannotCalculateChanges`, invalidate the affected cache/query result and perform a bounded resynchronization. Never infer ordering or timestamps from opaque JMAP states.[7]

Push notifications are hints to synchronize, not authoritative message content.

Use conditional state checks where supported for conflicting updates, and define user-visible conflict handling for offline drafts.

4.7 Retention, deletion, and portability

A deletion workflow must cover:

1. Mailbox objects.
2. Message and attachment blobs.
3. Search projections.
4. Client-cache invalidation.
5. Backup expiry under disclosed policy.
6. Legal holds, if applicable.
7. Auditable completion evidence.

Deletion from the visible inbox is not proof of physical deletion from every storage layer.

Export must preserve original message data and useful metadata. Test a round trip into a replacement service before claiming portability.

5. Integrated app-suite boundary

All apps share primitives; they do not share unrestricted access.

Shared primitive	Integration
Calendar	Courses, advising, campus events, mail invitations
Action model	Assignments, planning, career, support
Files/provenance	Study materials, portfolios, permitted attachments
Search	Results filtered by current ownership and permissions
Education graph	Account-to-school, course, role, and resource relationships
AI gateway	Source permissions, institutional policy, bounded actions
Audit/outbox	Controlled writes and recoverable downstream updates

Example: “Create an application task from this email” references the source message and requires approval. It does not copy the entire mailbox into Career or grant Career unrestricted mail access.

6. Phase 1–3 rollout roadmap

These phases are delivery gates, not calendar promises. Staffing and duration should follow the reconciliation audit.

Phase 1 — Identity, domains, and tenancy spine

Owner: platform/identity lead, with security and institution IT reviewers.

Deliverables:

- Canonical identity and migration decision.
- Verified domain inventory and routing.
- First-party OIDC clients and secure sessions.
- Recovery and safe account linking.
- One controlled school-federation pilot.
- Multi-school memberships and server-side policy enforcement.
- Signing-key, session-revocation, and incident runbooks.

Dependencies/blockers:

- Existing identity IDs and mappings must be understood.
- RLS/definer risks must be remediated or explicitly bounded.
- An institution must approve any real school federation.

Exit evidence:

- Wrong issuer/audience/redirect and replay attempts fail.
- Email collisions cannot merge accounts.
- Cross-tenant negative tests pass across API, database, files, search, and AI.
- Offboarding revokes one school without breaking unrelated access.
- Recovery and signing-key rotation drills pass.

Rollback: disable new federation connections and clients, revoke affected sessions, and retain the previous approved authentication path only if it remains secure.

Phase 2 — Mail pilot and shared productivity integration

Owner: messaging lead, with operations, security, and privacy owners.

Deliverables:

- Qualified mailbox engine and contractual review.
- Optional, invitation-only Semester Mail.
- Provisioning/reconciliation workflows.
- JMAP read, compose, draft, submission, and synchronization.
- Scoped search, export, retention, and abuse handling.
- User-approved mail-to-calendar/task integration.

Dependencies:

- Phase 1 identity and authorization gates passed.
- Verified mail-domain ownership and authentication.
- Support and abuse-response coverage.

Exit evidence:

- Provisioning retries do not create duplicate mailboxes.
- Unauthorized blob access and send-as attempts fail.
- Stale-state resynchronization works.
- Quotas, bounce handling, and suspension/appeal flows work.
- Restore and export/import drills pass.
- Personal mail remains inaccessible to school administrators by default.

Rollback: stop new provisioning; selectively disable outbound sending if required; preserve inbound delivery, existing data, and export access where safe. Never “roll back” by deleting pilot mailboxes.

Phase 3 — Multi-school suite and controlled ecosystem expansion

Owner: product/platform leads with domain and institutional owners.

Deliverables:

- Unified workspace switching and permitted cross-school personal planning.
- Shared Calendar, Files, Action Center, and search.
- Graduation/offboarding continuity.
- Approved institution-branded domains.
- Small reviewed partner OIDC pilot.
- Native-client continuity and tested offline synchronization.

Dependencies:

- Phase 2 operational stability and privacy evidence.
- Domain workflows have authoritative-source labels.
- Partner registration, consent, revocation, and incident processes exist.

Exit evidence:

- One institution cannot inspect another’s records or private workspace.
- Partner tokens cannot access unconsented resources.
- Account lifecycle changes reconcile across every enabled app.
- Offline changes cannot bypass expired permissions.
- Accessibility, support, restore, and migration tests cover end-to-end journeys.

Rollback: disable partner grants, custom-domain mappings, or individual features independently while keeping accounts, data access, and portability intact.

7. Readiness record

For each capability, maintain:

```
Owner
Canonical implementation
Dependencies
Designed / implemented / tested / deployed
Institution-authorized / production-enabled
Evidence links
Open risks
Activation approval
Rollback procedure
Last verification date
```

The supplied architecture snapshot documents unresolved security/tenancy issues and a “NO-GO · in progress” phase. That is not a live status claim, but it means phase 1 must reconcile those findings before public identity or mail activation. [2]

The resulting ecosystem is one durable Semester identity, multiple bounded school relationships, and a connected suite whose mail, learning, planning, and career workflows share infrastructure without erasing their different ownership and authority rules.

⁂

1. The Main Semester design system (2) copy/handoff/specifications/role-system-execution-prompt.pdf
2. The Main Semester design system (2) copy/uploads/Semester build Architecture.pdf
3. https://www.rfc-editor.org/info/rfc9700/
4. https://www.rfc-editor.org/info/rfc10017/
5. https://openid.net/specs/openid-connect-core-1_0.html
6. https://developers.google.com/identity/openid-connect/openid-connect
7. https://www.rfc-editor.org/info/rfc8620/
8. https://www.rfc-editor.org/info/rfc8621/
9. https://support.google.com/mail/answer/81126?hl=en
10. https://datatracker.ietf.org/doc/html/draft-ietf-jmap-core-08?ref=jevid.com
11. https://www.rfc-editor.org/subjects/oauth/
12. https://www.rfc-editor.org/status-changes/
13. https://www.rfc-editor.org/
14. https://www.rfc-editor.org/how-to-verify/
15. https://www.rfc-editor.org/series/rfc-errata/
16. https://oauth.net/2/oauth-best-practice/

17. https://workos.com/blog/oauth-best-practices

18. https://tex2e.github.io/rfc-translater/html/rfc8620.html

19. https://systemweakness.com/rfc-9700-for-oauth-2-0-explained-b51a4c6a1a09

20. https://queue.rfc-editor.org/about/